
Coding Exercise – Senior Software Engineer (.NET + UI + AI / Vibe Coding)

Our team heavily uses AI-assisted development (Vibe Coding, GitHub Copilot, ChatGPT). This exercise evaluates how you design a small .NET solution, build a simple UI, and work with AI tools.

You will be expected to **walk us through your code** during a virtual interview.

API To Use

Use the **Open-Meteo Historical Weather API** to fetch daily weather data.

Docs:

<https://open-meteo.com/en/docs/historical-weather-api>

Key points:

- No API key required, free for non-commercial use ([Open Meteo](#))
- Endpoint: /v1/archive
- You pass latitude, longitude, start_date, end_date, daily variables, and timezone ([Open Meteo](#))

Example pattern (for Dallas, TX; approx 32.78, -96.8):

```
https://archive-api.open-meteo.com/v1/archive?latitude=32.78&longitude=-96.8&start_date=2021-02-27&end_date=2021-02-27&daily=temperature_2m_max,temperature_2m_min,precipitation_sum&timezone=auto
```

You may adapt variables as needed.

Functional Requirements

1. Read Input Dates

Create a file named dates.txt in your project, containing:

- 02/27/2021
- June 2, 2022
- Jul-13-2020

- April 31, 2022

Your backend must:

- Read all dates from dates.txt
 - Parse multiple formats into ISO (yyyy-MM-dd)
 - Treat invalid dates (for example, April 31) as invalid
 - Handle invalid dates gracefully (no crash; record an error)
-

2. Fetch Historical Weather From Open-Meteo

For each **valid** date:

- Call the Open-Meteo **Historical Weather** API for **Dallas, TX**
- Request at least these **daily** fields:
 - Minimum temperature
 - Maximum temperature
 - Precipitation sum

Handle:

- Network/API failures
 - Empty or missing data
 - Date ranges that return no data
-

3. Store Results Locally

For each valid date, store the result as JSON under a folder such as:

- weather-data/2021-02-27.json
- weather-data/2022-06-02.json

Avoid unnecessary repeat calls if data for that date was already stored (for example, simple existence check).

4. Backend API

Expose at least one endpoint, for example:

- GET /api/weather

This should return a list of entries, one per **valid** date, including:

- Date (normalized)
- Min temperature
- Max temperature
- Precipitation
- Any status or error message if something went wrong for that date

You may structure the response as you see fit, as long as it is sensible and consistent.

5. UI Requirements (Mandatory)

Build a simple web UI using one of:

- Angular
- React
- Blazor
- Razor Pages

The UI must:

1. Call your backend endpoint to load the weather data.
2. Display the list in a table or list with:
 - Date
 - Min temp
 - Max temp
 - Precipitation
3. Show a **loading** state while fetching.
4. Show a clear **error** message if the API call fails.

5. Include at least one small interaction, such as:

- Sort by temperature or date
- Filter by date range or minimum temperature
- Click a row to show details for that date

Visual design can be minimal; clarity matters more than styling.

AI / Vibe Coding Usage (Required)

You are encouraged to use AI tools (Vibe Coding, Copilot, ChatGPT, etc.).

Add a file called **AI_NOTES.md** in the repo root that explains:

1. Which AI tool(s) you used.
2. Two or three prompts that helped you the most.
3. One concrete example where the AI's suggestion was wrong or not ideal, and how you detected and corrected it.
4. Parts of the solution where you chose to write code yourself rather than rely on AI, and why.

We are evaluating **how you work with AI**, not whether you avoid it.

Tech Expectations

- Backend in **C#** using **.NET 6 or later**.
 - Frontend in Angular, React, Blazor, or Razor Pages.
 - Use async/await appropriately for HTTP calls.
 - Do not hardcode secrets (none are required here, but treat config cleanly).
-

Acceptance Criteria

Your submission is complete when:

1. You send a **public GitHub repository link**.
2. The solution builds and runs locally using steps in your README.md.

3. The app:

- Reads and parses dates.txt
- Handles invalid dates without crashing
- Calls Open-Meteo for each valid date and collects daily weather data
- Stores results under a weather-data folder (or similar)
- Exposes a backend endpoint returning aggregated weather data
- Displays that data in a UI that meets the requirements above

4. The repo contains:

- README.md describing:
 - How to run the backend
 - How to run the UI
 - Any assumptions
- AI_NOTES.md describing your AI usage

5. There are multiple **small commits** that show your workflow (not just one big dump commit).

Please aim to spend about **2–3 hours** on this exercise. We care more about structure, clarity, and your reasoning than about completeness.

Code Walkthrough (Interview)

During the virtual interview, we will ask you to:

- Share your screen
- Walk through the project structure (solution, projects, main folders)
- Explain how you:
 - Parsed and validated dates
 - Designed the API integration
 - Structured error handling and storage
 - Built the UI and handled loading/error states

- Used AI tools and corrected them when needed
- Describe what you would improve if this were a production feature.